

The root user: centralised root access and break-glass

Promised in modules 11 and 22 – the capability that lets you delete root credentials and still recover.

The safest root user is one that **does not exist** – and you can now have that without losing recovery.

What only the root user can do

- ◆ Close the account; change its name, email or root password; change the support plan.
- ◆ **Unlock a resource policy that denies every principal** – an S3 bucket or SQS queue policy locked against everyone.
- ◆ Restore IAM permissions after an admin removes their own. Register as a Marketplace seller. Sign up for GovCloud.
- ◆ Everything else is ordinary IAM. If a task is on no such list, root is not the answer.
- ◆ The lock-out case is why root credentials survived so long. It has no substitute.

ON THE EXAM

Nobody in the account can edit a deny-all policy, because the policy denies them.

Centralised root access: two features

- ◆ **Root credentials management** – remove and prevent root credentials in member accounts.
- ◆ **Root sessions** – short, task-scoped root access when you genuinely need it.
- ◆ Enabled separately. Check both with **iam list-organizations-features**.
- ◆ **New member accounts are created with no root credentials**. Secure by default.
- ◆ To delete existing credentials, **deactivate MFA devices first**.

ON THE EXAM

Take the credential away, keep the capability centrally.

The four task policies

POLICY ARN SUFFIX	SCOPED TO
<code>root-task/S3UnlockBucketPolicy</code>	Deleting an S3 bucket policy that locks everyone out
<code>root-task/IAMDeleteRootUserCredentials</code>	Deleting passwords, access keys, certificates and MFA devices
<code>root-task/IAMCreateRootUserPassword</code>	Creating a member root login profile
<code>root-task/IAMAuditRootUserCredentials</code>	Reviewing root credentials

ON THE EXAM

A root session is not full root. It is scoped to exactly one of these managed policies, and nothing else.

How a root session actually works

- ◆ Call **sts:AssumeRoot** from the **management account or the IAM delegated administrator**, naming a target member account and one task policy.
- ◆ **You cannot use root user credentials to make this call.** An IAM principal with sts:AssumeRoot permission makes it.
- ◆ **Maximum and default duration is 900 seconds – fifteen minutes.** It cannot be extended.
- ◆ **The STS global endpoint is not supported.** AssumeRoot must go to a Regional STS endpoint.
- ◆ The response carries **Sourcidentity** – the calling principal – so CloudTrail shows which human ran the privileged task.

ON THE EXAM

Sourcidentity from module 21 appearing again: an immutable record of who, attached to the most privileged session AWS offers.

What still applies during a root session

- ◆ **Service control policies remain effective.** A root session is not exempt from your ceilings.
- ◆ **VPC endpoint policies apply too**, so root sessions can be restricted to expected networks.
- ◆ An SCP denying what the task needs will block the session. Review deny-all-root SCPs first.
- ◆ Module 11: SCPs **do** apply to member root. The **management account** root is the exempt one.
- ◆ Short, scoped, logged – **and** still inside your guardrails.

ON THE EXAM

Enabling root sessions does not create a hole through your SCPs.

The management account root, and monitoring

- ◆ You cannot remove the management account's root credentials. It stays a real root user.
- ◆ So: **phishing-resistant MFA**, a break-glass procedure, sealed credentials, alarms on use.
- ◆ **The root-usage alarm needs updating.** Once member credentials are gone, ConsoleLogin events stop.
- ◆ **Alarm on sts:AssumeRoot instead** – that is where privileged access now happens.
- ◆ Every call is in CloudTrail with **SourceIdentity**. A better trail than root sign-in ever was.

ON THE EXAM

A security improvement silenced an alarm. Silence looks like compliance until you check why.

An engineer in a member account applies a bucket policy that denies s3:* to Principal "*" with no exceptions, intending to lock the bucket down temporarily. Nobody in the account can now read, write, or edit the bucket policy - including the account's administrators.

The organisation has centralised root access enabled and no root credentials in member accounts.

How should this be recovered?

- ☐ A sts:AssumeRoot from the management account with the S3UnlockBucketPolicy task policy
- ☐ B Recreate the member account's root password and sign in as root
- ☐ C Attach an SCP allowing s3:DeleteBucketPolicy to the account's OU
- ☐ D Delete the bucket and restore its contents from a backup

◆ PAUSE AND DECIDE

How should this be recovered?

- A

 AssumeRoot with S3UnlockBucketPolicy
Correct. This is the canonical use case – a 15-minute session scoped to exactly one action, from the management account, fully logged with SourceIdentity.
- B

 Recreate the root password and sign in
Possible via IAMCreateRootUserPassword, and it recreates the credential the organisation deliberately removed to solve a problem with a purpose-built session type.
- C

 An SCP allowing s3:DeleteBucketPolicy
The module 19 rule: an SCP restricts and never grants. It also would not override an explicit Deny in the bucket policy.
- D

 Delete the bucket and restore
The bucket policy denies everything, including deleting the bucket. And destroying data to fix a policy error is the wrong instinct.

THE SIGNAL

A deny-all resource policy is the textbook root task. It is why root sessions exist, and why S3UnlockBucketPolicy is one of only four.

A company enables centralised root access, removes root credentials from all 140 member accounts, and confirms new accounts are created without them. Its CIS-aligned CloudWatch alarm on root user ConsoleLogin events has produced no alerts in four months. The security lead reports this to the board as evidence that root usage has ceased.

What is the problem with that conclusion?

- ☐ A Root ConsoleLogin events are not recorded by CloudTrail, so the alarm never worked
- ☐ B Nothing – no alerts correctly means no root usage
- ☐ C The alarm should be on the management account only, where root still exists
- ☐ D There are no member root users left to sign in; privileged access now happens through sts:AssumeRoot, which the alarm does not watch

◆ PAUSE AND DECIDE

What is the problem with that conclusion?

- A

 Root ConsoleLogin is not in CloudTrail
It is – root activity is logged like any other principal, and alarming on it is a standard benchmark control. The alarm worked; its subject disappeared.
- B

 No alerts means no root usage
The conclusion being tested. The alarm watches an event that can no longer occur, so its silence carries no information at all.
- C

 Alarm on the management account only
Half right – the management account root does still exist and needs alarming. It misses that member-account privileged access moved rather than stopped.
- D

 No member root users remain; privileged access is now AssumeRoot
Correct. The control needs to follow the capability. Alarm on sts:AssumeRoot, which carries SourceIdentity and is a better audit trail than root sign-in.

THE SIGNAL

A security improvement silenced a detective control. Both facts are true, and only one of them was noticed.

During an incident, an administrator signs in to the organisation's management account as the root user and attempts to run `sts:AssumeRoot` against a member account to delete its root credentials. The call is rejected.

Centralised root access is enabled, both features are on, and the target account is a member of the organisation.

Why was the call rejected?

- ☐ A IAMDeleteRootUserCredentials can only be used from the delegated administrator, not the management account
- ☐ B The target account must have no MFA devices before AssumeRoot can be called
- ☐ C You cannot use root user credentials to call AssumeRoot; an IAM principal with `sts:AssumeRoot` must make the call
- ☐ D AssumeRoot requires a duration of at least 3600 seconds, which exceeds the root session limit

◆ PAUSE AND DECIDE

Why was the call rejected?

- A

 Delegated administrator only
AWS documents both the management account and the IAM delegated administrator as valid callers. The account is not the problem; the identity is.
- B

 Target must have no MFA devices first
MFA devices must be deactivated before deleting credentials – a step inside the session, not a precondition for starting one.
- C

 Root credentials cannot call AssumeRoot
Correct. AWS states it explicitly. The call must come from an IAM principal holding sts:AssumeRoot – you cannot use root to obtain root.
- D

 Minimum duration of 3600 seconds
Backwards. 900 seconds is the maximum and the default, and there is no minimum that exceeds it.

THE SIGNAL

Signing in as root to run AssumeRoot is the one identity that cannot do it. The whole design assumes root is not in use.

An organisation has long applied an SCP at the root denying all actions when `aws:PrincipalIsAWSService` is false and the principal is a root user, to stop member root usage entirely. It now enables centralised root access and root sessions.

Every attempted root session against a member account fails, even though the caller holds `sts:AssumeRoot` and the task policy is correct.

Why, and what should the organisation do?

- A Root sessions require the SCP to be detached from the root and reattached to each OU
- B SCPs remain effective during root sessions; the SCP must permit the actions the task policy needs
- C Root sessions bypass SCPs, so the failure must be a permissions issue in the management account
- D Centralised root access cannot be used in organisations with root-denying SCPs; remove the SCP entirely

◆ PAUSE AND DECIDE

Why, and what should the organisation do?

- A Detach from root and reattach per OU
Placement changes which accounts are in scope, not whether the policy applies during a root session. It would still deny the same actions.
- C Root sessions bypass SCPs
The opposite of the documented behaviour, and if it were true the feature would be a hole straight through every organisational ceiling.

- B SCPs remain effective; permit what the task policy needs
Correct. AWS documents that SCPs, VPC endpoint policies and other policies remain effective during root sessions - which is a feature, not a bug.
- D Remove the SCP entirely
Overreacts. The fix is a narrow exception for the specific task actions, not abandoning a control that has been working.

THE SIGNAL

That SCPs still apply is what makes root sessions safe to enable. The blanket deny needs a narrow exception, not removal.

Six things to carry forward

- ◆ **Two separate features:** root credentials management, and root sessions. Check both with list-organizations-features.
- ◆ **Four task policies, one per session.** A root session is never full root.
- ◆ **15 minutes maximum, Regional STS endpoint only, and root credentials cannot make the call.**
- ◆ **SCPs remain effective during a root session.** A blanket root deny will block it.
- ◆ **You cannot remove the management account's root credentials.** It keeps the full break-glass treatment.
- ◆ **Move your root alarm to sts:AssumeRoot.** ConsoleLogin goes silent once member credentials are gone.

ON THE EXAM

Next: deploying securely – StackSets, CloudFormation Guard, Service Catalog and RAM.